Проектирование безопасной микросервисной архитектуры

Проектирование микросервисной архитектуры для системы «Электронная библиотека»

Домашнее задание №5

ВЫПОЛНИЛ: ШУМКОВ СЕРГЕЙ
ДАТА: 30 ИЮЛЯ 2026 Г.

Содержание

- Анализ угроз
- Описание аутентификации и авторизации
- Проектирование защищенного контура
- Основные сценарии взаимодействия

Исходные данные

- Приобретён готовый каталог книг в формате EPUB (файлы + метаданные).
- Есть возможность докупать книги, поступающие ZIP-архивами (по несколько книг в архиве).
- **Технические ограничения:**
- Файлы хранятся в собственном S3-хранилище, метаданные - в БД.
- Для скачивания книг создаются временные ссылки.
- **Ключевые бизнес-процессы**
- Пополнение каталога контент-менеджером.
- Поиск и скачивание книг читателем.
- После прочтения читатель может оставить отзыв и поставить оценку. Отзыв проходит премодерацию.
- Читатель создаёт обращение (тикет), сотрудник техподдержки отвечает на обращение, созданное читателем.

Анализ угроз

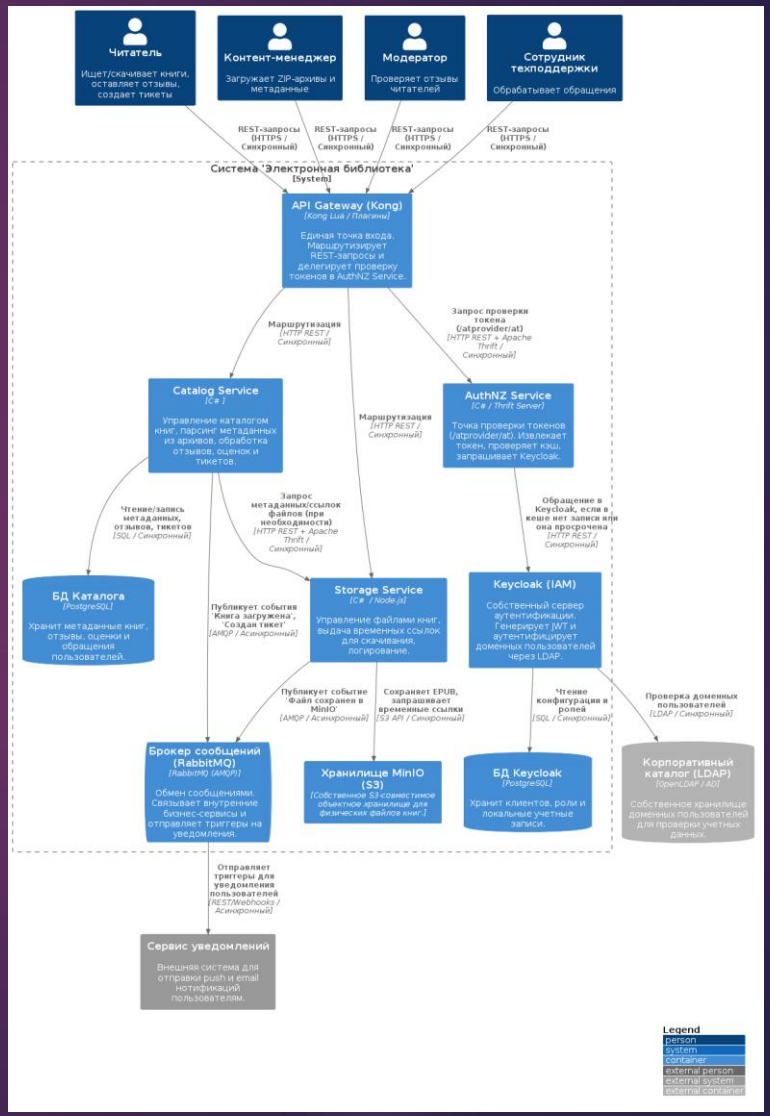
№	Угроза	Уровень угрозы	Где возникает	Возможные последствия
1	Кража учетных данных читателя	Пользовательский уровень	Интерфейс читателя	Несанкционированный доступ к аккаунту, утечка личных данных
2	Перехват токенов	Уровень API	Между клиентом и API Gateway	Компрометация сессии читателя, скачивание платных книг
3	DDOS-атака	Уровень API	Снаружи на и API Gateway	Недоступность сервиса для пользователей
4	Подмена в Payload JWT-токена данных пользователя. Например смена роли Читатель на Админ	Уровень сервисов	Межсервисное взаимодействие	Выполнение действий от имени Администратора
5	Кража файлов книг из хранилища	Уровень данных	Хранилище MinIO S3	Скачивание платных книг
6	SQL-иньекции	Уровень данных	СУБД PostgreSQL	Изменение оценок, удаление отзывов.
7	Инсайдерская атака	Инфраструктурный уровень	Окружение k8s	Компрометация мастер-ключей шифрования (keycloak или БД)

Описание аутентификации и авторизации

- Подход к аутентификации: OpenID Connect на базе OAuth 2.0 + JWT.
- Обоснование:
  - OIDC предоставляет готовый слой идентификации поверх OAuth 2.0.
  - Совместимость с API Gateway (Kong).
  - Подпись токена гарантирует, что злоумышленник не сможет подделать данные внутри токена, так как у него нет приватного ключа Keycloak.
- Модель авторизации: Комбинированная (RBAC + ABAC)
  - RBAC – необходим для разделения базовых ролей (Читатель, Техподдержка и т.д.)
  - ABAC – необходим для гибких бизнес-правил (например, подписка «Премиум»)

Проектирование защищенного контура

- Провайдер идентификации: Keycloak для централизованного хранения учетных записей.
- API Gateway: Точка входа, реализующая защиту от DDoS и проверку TLS.
- Механизм аутентификации: Проверка валидности подписи JWT токена в AuthNZ Service с кэшированием в Redis.
- Защищенные каналы: Внешний HTTPS от клиента до шлюза.



Основные сценарии взаимодействия

- 1. Процесс логина и получение токена. :
  - Пользователь отправляет POST /auth/login (login/password) на API Gateway.
  - API Gateway отправляет запрос в Keycloak.
  - Keycloak проверяет хэш пароля. При успехе генерирует пару: Access Token и Refresh Token.
  - Токены возвращаются Пользователю.
  - Клиент сохраняет Access Token и Refresh Token.
- 2. Доступ к сервису и проверка токена:
  - Пользователь делает запрос GET /books/1 с заголовком Authorization: Bearer <access_token>.
  - API Gateway перехватывает запрос и отправляет токен в AuthNZ Service.
  - AuthNZ проверяет токен в Redis. Если токен валиден, шлюз обогащает заголовки (X-User-Id, X-User-Roles) и передает запрос в Catalog Service.

- 3. Отказ в доступе
  - 401 Unauthorized: Токен просрочен или подпись неверна. API Gateway сразу отклоняет запрос, не нагружая бэкенд.
  - 403 Forbidden: Читатель пытается вызвать POST /books (добавление книги). Сервис видит в заголовке роль Reader вместо ContentManager и возвращает ошибку 403.
- 4. Refresh токена
  - Клиент получает ошибку 401 Unauthorized (Токен просрочен).
  - Клиент автоматически отправляет POST /auth/refresh, прикрепляя Refresh Token.
  - API Gateway направляет запрос в Keycloak.
  - Keycloak валидирует Refresh токен.
  - Keycloak возвращает новый Access Token.
  - Клиент повторяет исходный запрос.